

MISC-04 — Research-Backed Data & Model Pack

Fills the placeholders in the Technical Workflow with real, sourced material

This document answers, with real sources, the four gaps flagged earlier: (1) where real data actually comes from, (2) which district/event has abundant documentation, (3) what weights real flood-risk research actually uses, and (4) how to fine-tune the model responsibly. Use this alongside `misc04-technical-workflow.md` — it supplies the real inputs that document's formulas expect.

1. Chosen District & Event: Wayanad, Kerala — August 2018 Floods

Why this specific pick, not just "a district":

- Wayanad is a real, well-documented district (Kerala), consisting of **3 taluks — Mananthavady, Sulthan Bathery, Vythiri — and 49 revenue villages**, headquartered at Kalpetta. District area 2,132 sq.km, population ~817,420.
- During the August 2018 Kerala floods, Wayanad received rainfall **299% above normal** for the period, triggering landslides and **snapping road links between Wayanad and the rest of Kerala** — this is a direct, real match to the PS's "affected settlements + road accessibility" framing, not a hypothetical.
- The India Meteorological Department's station at **Mananthavadi recorded 305 mm of rainfall in a single day (8–9 August 2018)**, against a district monsoon total well above normal — this is a real, citable number, not invented.
- Kerala's own flood report (Central Water Commission, via Kerala SDMA) documents 1-day rainfall figures for several real stations during this event: Nilambur (Malappuram, neighboring district) 398 mm, Mananthavadi (Wayanad) 305 mm, plus Peermade, Munnar and Devikulam readings of 211–255 mm — giving you a real, escalating multi-station rainfall record to build your 8 timesteps from, instead of inventing a curve.
- The river relevant to Wayanad is the **Kabani (Kabini) river** — real, nameable, usable for your distance-to-river feature.

This is a stronger pitch narrative than a generic district because you can say, on camera: *"We used the real rainfall record from the August 2018 Wayanad floods, the same event that cut off road access to the district — this isn't simulated, it's the real historical event, replayed."*

Real settlements to use (real village/town names, not invented)

Pulled from Wayanad's actual taluk/village structure — pick ~15-20 for your prototype, spread across all 3 taluks so risk variation looks geographically real:

- **Mananthavady taluk:** Mananthavady town, Thirunelli, Thavinal, Panamaram
- **Sulthan Bathery taluk:** Sulthan Bathery, Ambalavayal, Nenmeni, Noolpuzha
- **Vythiri taluk:** Kalpetta, Vythiri, Meppadi, Lakkidi, Chundale, Kaniyambetta, Padinjarathara

For each, look up real lat/lng via OpenStreetMap search (free, no scraping needed — just search the name) and real elevation via the Open-Elevation API using that coordinate. Do not fabricate elevation numbers — Wayanad's terrain genuinely varies from ~700m to 2100m above sea level, so real elevation differences between these villages will produce genuinely different risk outputs, which is exactly what your acceptance criteria require.

Where to get the real rainfall/district datasets (direct sources)

Dataset	What it gives you	Source
Kerala Floods 2018 (Kaggle, imdevskp)	Rainfall, fatalities, relief camps, warnings during the event	kaggle.com/datasets/imdevskp/kerala-floods-2018
Rainfall in Kerala 1901-2017 (Kaggle / data.gov.in)	Monthly + annual subdivision rainfall, long historical baseline to compute "normal" for comparison	data.gov.in — sub-divisional monthly rainfall 1901-2017 dataset
India-WRIS District Daily Rainfall	Real daily rainfall by district (state_code/district_code columns), CSV, actively maintained	indiawris.gov.in/wris/#/rainfall
Kerala SDMA / CWC Post-Flood Report (2018)	Real station-level 1-day/3-day/5-day rainfall figures for the actual event, used above	sdma.kerala.gov.in (CWC Report on Kerala Floods, PDF)
Open-Elevation API	Real elevation per lat/long, free, no key required	open-elevation.com/api
OpenStreetMap Nominatim	Real coordinates for village/town names	nominatim.openstreetmap.org

This is enough to build every field in `state.json`'s `settlements[]` array from a real, traceable source — nothing left as a placeholder by the time you demo.

2. Real Feature Weights (from published flood-susceptibility research)

You asked what weights existing solutions actually use — here's what the literature says, so your formula isn't guessed:

- A Random Forest flood-susceptibility study on the **Tajan watershed** found, in order of importance: **elevation (highest impact)**, **distance from river (second)**, **slope (third)** — with elevation and distance-from-river together accounting for the clear majority of predictive weight among terrain factors.

- A **Kerala-specific study (Ernakulam district, NITK/IEEE 2022)** used twelve flood-conditioning factors — elevation, slope, curvature, rainfall, distance to river, drainage density, land use, and others — confirming that **elevation, rainfall, and distance-to-river are the three factors used most consistently in Kerala flood models specifically**, which matches your district choice.
- A regional feature-ranking study (Pahang, Malaysia, PLoS ONE 2024) found **rainfall and elevation were consistently ranked as the two primary attributes**, ahead of slope, geology and land-use factors.
- A Himachal Pradesh SHAP-based study found elevation alone carried roughly **3x the predictive weight of rainfall** in mountainous terrain specifically, though rainfall's relative importance rises in flatter, monsoon-dominated regions like Kerala — relevant since Wayanad is hilly but monsoon-driven, so both matter roughly comparably.

Translated into a justified, documented formula for your 4-feature model (rainfall, elevation, distance-to-river, historical-flood-flag — matching what you can realistically source in 2 days):

```
risk_score_proxy_label_rule:
  y = 1 if (rainfall_mm_normalized * 0.35
    + (1 - elevation_normalized) * 0.30
    + (1 - distance_to_river_normalized) * 0.20
    + historical_flood_flag * 0.15) > 0.5
```

Where each `_normalized` term is min-max scaled 0-1 across your settlement set. This weighting is not arbitrary: rainfall and elevation are weighted highest (0.35 / 0.30) because they were the two most consistently top-ranked factors across the Kerala-specific and Pahang studies cited above; distance-to-river next (0.20), consistent with its second/third-place ranking in the Tajan watershed study; historical flood occurrence last (0.15) as a real but secondary signal. **State this reasoning explicitly in your README and PPT** — "our weights are informed by published flood-susceptibility literature, not arbitrary" is a strong, defensible line if a judge asks.

Use this rule only to generate the **proxy training label** (Section 2.3 of the technical workflow) if you can't find verified per-settlement flood-occurrence ground truth in time. Once trained, let the actual Random Forest / logistic regression learn its own internal feature importances from the data — report those too, and compare them to the literature values above as a validation step ("our trained model's feature importances broadly agree with published Kerala flood studies, per rainfall/elevation dominance") — this is a genuinely strong, real validation claim, not a fabricated one.

3. Model Fine-Tuning Guidance (what to actually tune, and why)

Since you'll fine-tune the model yourselves, tune these specific parameters — don't just run defaults:

If using Random Forest (recommended — handles the small, noisy dataset you'll have better than logistic regression):

- `n_estimators`: start at 100; with only ~15-20 settlements $\times$ 8 timesteps (~120-160 rows), going much higher won't help and risks overfitting noise.
- `max_depth`: cap at 4-6. With this little data, an unconstrained tree will memorize rather than generalize — shallow trees are more defensible when a judge asks "how do we know this isn't just memorizing your 20 villages."
- `min_samples_leaf`: set to 3-5 to prevent single-row leaf nodes, given your small dataset.
- `class_weight='balanced'`: important if your proxy-label rule produces an imbalanced split (e.g. far more "low risk" than "high risk" rows) — otherwise the model will just learn to predict "low risk" always and look artificially accurate.
- After training, print `model.feature_importances_` and compare it against Section 2's literature values — this comparison is your strongest "we validated this isn't random" evidence.

If using Logistic Regression instead (simpler, easier to explain to judges):

- `C` (inverse regularization strength): start at `C=1.0`, try `0.1` and `10` and compare — with small data, stronger regularization (`C=0.1`) often generalizes better.
- Standardize features first (`StandardScaler`) — required for logistic regression to weight features meaningfully; skip this and rainfall (large numbers, e.g. 300mm) will dominate elevation (also large, e.g. 900m) for the wrong reason (scale, not real importance).
- The fitted coefficients (`model.coef_`) are directly your feature weights after training — report these next to the literature-informed weights from Section 2 as your validation comparison.

Either way — confidence score derivation (already specified in the technical workflow): $\text{confidence} = \frac{\text{abs}(\text{predicted_probability} - 0.5)}{2}$. Tune nothing here; it's a deterministic transform of the model's own probability output, which is what makes it a real, not decorative, uncertainty measure.

4. Answering the Four Disadvantages Directly

Original concern	Concrete resolution
"Doc has placeholders it can't fill on its own"	Section 1 above replaces every placeholder — real district (Wayanad), real event (Aug 2018), real villages, real rainfall stations, real river, real dataset links.
"AI writes code that runs, not code that's correct"	Section 3 gives exact hyperparameters and the specific check (compare trained feature importances against cited literature values) that proves correctness, not just execution.
"Integration breaks silently across modules"	The <code>state.json</code> contract (in the technical workflow doc) is now filled with real field-level meaning from this document — every member builds against the same real settlement list and real timestep rainfall values, not independently-invented placeholder data, which is what usually causes integration mismatches.
"AI doesn't know what 'good enough for judges' looks like"	This document gives you the actual sentence to say on camera ("real rainfall record from the August 2018 Wayanad floods, replayed") and the actual validation line ("our model's feature importances agree with published Kerala flood studies") — these are the lines that make judges trust the build, and no coding tool will generate them for you.

What's still on you and your teammates, even with this document: actually pulling the CSVs from the links above, running the elevation lookups, training the model and checking the numbers look sane, and rehearsing the pitch lines above until they sound natural, not read off a slide.